

COMP2012 LAC Practice Pack

Four brand new 2-hour practice papers with detailed mark schemes

Languages, Automata, CFGs, PDAs, Parsers, Turing Machines, and the Chomsky hierarchy

Suggested timing: 2 hours per paper
Marks: 100 marks per paper
Style: Old-paper style, but with fresh questions and broader current-note coverage
Notation: + means union/choice, juxtaposition means concatenation, ε means empty word

Contents

How to use this pack	2
Practice Paper A - Core Mock	3
Practice Paper B - Parser and PDA Mock	6
Practice Paper C - Pumping and Halting Mock	8
Practice Paper D - Synthesis Mock	10
Detailed Mark Schemes	12
Mark Scheme: Practice Paper A	13
Mark Scheme: Practice Paper B	15
Mark Scheme: Practice Paper C	17
Mark Scheme: Practice Paper D	18

How to use this pack

Do each paper under timed conditions first. Do not look at the mark scheme until afterwards. The questions are intentionally slightly broader than the old paper: Paper A is closest to the old-paper core, while Papers B-D increasingly test the under-revised danger zones such as LL(1) parsing, PDA mechanics, and Turing-machine/decidability distinctions.

Paper	Main emphasis
Paper A	Core DFA/NFA, regexes, regularity, CFG ambiguity, Chomsky hierarchy
Paper B	Subset construction, LL(1) parsing, PDA design, TM high-level behaviour
Paper C	Pumping lemma, palindromes, halting problem, decidability/r.e. languages
Paper D	Mixed synthesis paper: language algebra, regex grammar ambiguity, hierarchy/closure facts

Exam technique rule: if asked to show a language is regular, give a DFA, NFA, or regex. If asked to show it is not regular, use a non-regularity argument such as the pumping lemma or closure contradiction. If asked about ambiguity, you need the same word with two different parse structures, not merely two different derivation orders.

Practice Paper A - Core Mock

Time allowed: 2 hours. Total: 100 marks.

Answer all questions. Show enough working for non-multiple-choice questions.

Question 1 - Multiple choice / True-False [25 marks]

For each subquestion, mark every statement True or False. There may be any number of true statements.

1. Automaton A1 over $\{0, 1\}$ [5 marks]

A1 has states $\{s, z, o\}$, start state s , final states $\{o\}$, and transition table:

	0	1
s	s	o
z	s	o
o	z	o

- (a) A1 is a DFA.
- (b) A1 is an NFA but not a DFA.
- (c) $001 \in L(A1)$.
- (d) $\varepsilon \in L(A1)$.
- (e) $L(A1)$ is exactly the set of binary words ending in 1.

2. Automaton A2 over $\{0, 1\}$ [5 marks]

A2 has states $\{p, q, r\}$, initial state p , final state r , and transition function:

	0	1
p	$\{p\}$	$\{p, q\}$
q	$\{r\}$	$\emptyset$
r	$\{r\}$	$\{r\}$

- (a) A2 is an NFA but not a DFA.
- (b) A2 is a DFA.
- (c) $110 \in L(A2)$.
- (d) $111 \in L(A2)$.
- (e) A2 accepts every word containing the substring 10.

3. Basic language theory [5 marks]

- (a) A language over Σ is a set of words over Σ .
- (b) A word is a set of symbols.
- (c) DFAs and NFAs recognise the same class of languages.
- (d) Deterministic PDAs and nondeterministic PDAs recognise the same class of languages.
- (e) Deterministic and nondeterministic Turing machines recognise the same class of languages.

4. CFG membership [5 marks]

Let G_1 over $\{a, b, c\}$ have productions

$$S \rightarrow aSb \mid C, \quad C \rightarrow cC \mid \varepsilon.$$

Which words are in $L(G_1)$?

- (a) ε
- (b) acb

- (c) *aaccbb*
- (d) *abcc*
- (e) *aacbb*

5. Halting and decidability**[5 marks]**

- (a) The Halting Problem is recursively enumerable / semi-decidable.
- (b) The complement of the Halting Problem is recursively enumerable / semi-decidable.
- (c) A decider for a language halts on every input.
- (d) Every recursively enumerable language is decidable.
- (e) The Halting Problem can be expressed as a language of encodings.

Question 2 - Regular expressions and regular languages**[25 marks]**

Let $\Sigma = \{a, b, c\}$. Give regular expressions for the following languages. Use $+$ for union and write concatenation by placing expressions next to each other. **[20 marks]**

- (a) Words containing only *cs*.
- (b) Words containing the substring *bc*.
- (c) Words containing an even number of *as*.
- (d) Words starting with *a* and ending with *b*.
- (e) Words containing no *cs*.
- (f) Words containing exactly one *b*.
- (g) Words not containing the substring *bb*.
- (h) Words of length at least 2.
- (i) Words of the form some *as*, then some *bs*, then some *cs*.
- (j) Words ending in either *abc* or *ac*.

Now classify each language as regular or not regular.

[5 marks]

- (i) $L_1 = \{w \mid \#a(w) \equiv \#b(w) \pmod{2}\}$.
- (ii) $L_2 = \{a^n b^n \mid n \in \mathbb{N}\}$.
- (iii) $L_3 = \{[a], [b, c], [a, b, c]\}$.
- (iv) $L_4 = \Sigma^* \setminus L_3$.
- (v) $L_5 = \{ww \mid w \in \Sigma^*\}$.

Question 3 - CFGs, derivations, and ambiguity**[25 marks]**

Consider the ambiguous expression grammar over tokens $a, +, *, (,)$:

$$E \rightarrow E + E \mid E * E \mid (E) \mid a.$$

- (a) Give a derivation showing that $a + a * a \in L(G)$. **[8 marks]**
- (b) Explain why this grammar is ambiguous. Your answer must identify one word and two different parse structures. **[8 marks]**
- (c) Write an unambiguous grammar for the same expression language where $*$ binds more tightly than $+$. **[6 marks]**
- (d) Briefly explain how your grammar enforces precedence. **[3 marks]**

Question 4 - Chomsky hierarchy and Turing-machine concepts**[25 marks]**

- (a) State the four levels of the Chomsky hierarchy, with the associated language class, automaton model, and grammar formalism. **[8 marks]**

- (b) Give examples of languages that separate the levels: one regular language, one context-free but not regular language, one context-sensitive but not context-free language, and one recursively enumerable undecidable language. **[8 marks]**
- (c) Explain the difference between decidable and recursively enumerable languages. **[4 marks]**
- (d) Why are decidable languages not usually treated as a proper level of the Chomsky hierarchy? **[3 marks]**
- (e) State one reason why Turing machines are more powerful than PDAs. **[2 marks]**

Practice Paper B - Parser and PDA Mock

Time allowed: 2 hours. Total: 100 marks.

Answer all questions.

Question 1 - Multiple choice / True-False [25 marks]

1. DFA D over $\{a, b\}$ [5 marks]

D has four states tracking parity of the number of a s and b s: EE is start and final, EO , OE , OO have their obvious meanings. Reading a toggles the first parity; reading b toggles the second.

- (a) $\varepsilon \in L(D)$.
- (b) $abba \in L(D)$.
- (c) $aba \in L(D)$.
- (d) $L(D)$ is the set of words with an even number of a s and an even number of b s.
- (e) Recognising this language requires infinitely many states.

2. NFA C over $\{a, b\}$ [5 marks]

C has states p, q, r, f , start p , final f , transitions $p \xrightarrow{a,b} p$, $p \xrightarrow{a} q$, $q \xrightarrow{b} r$, $r \xrightarrow{a} f$, and $f \xrightarrow{a,b} f$.

- (a) C is nondeterministic.
- (b) C accepts exactly the words containing substring aba .
- (c) $ababa \in L(C)$.
- (d) $abb \in L(C)$.
- (e) Every DFA-equivalent version of C must have exactly four states.

3. Language operations [5 marks]

- (a) The language $\{\varepsilon\}$ contains exactly the empty word.
- (b) $\emptyset^* = \{\varepsilon\}$.
- (c) $L \cdot \emptyset = \emptyset$ for every language L .
- (d) $L \cdot K = K \cdot L$ for all languages L, K .
- (e) $(L \cup K)^* = L^* \cup K^*$ for all languages L, K .

4. PDA facts [5 marks]

- (a) A PDA is a finite automaton equipped with a stack.
- (b) Every context-free language is recognised by some nondeterministic PDA.
- (c) Deterministic and nondeterministic PDAs recognise exactly the same languages.
- (d) Palindromes over $\{a, b\}$ can be recognised by a nondeterministic PDA.
- (e) A single-stack PDA recognises $\{a^n b^n c^n \mid n \geq 0\}$.

5. Parser facts [5 marks]

- (a) Left recursion can make a naive recursive-descent parser loop forever.
- (b) LL(1) parsing uses one token of lookahead to choose productions.
- (c) $First(\alpha)$ contains terminals that can begin strings derived from α .
- (d) $Follow(A)$ contains terminals that appear immediately before A .
- (e) Eliminating left recursion can preserve the language while changing the grammar structure.

Question 2 - NFA to DFA and regex construction**[25 marks]**

- (a) Use the subset construction on the following NFA over $\{a, b\}$. It has states p, q, r , start state p , final state r , and transitions:

$$\delta(p, a) = \{p, q\}, \quad \delta(p, b) = \{p\}, \quad \delta(q, a) = \emptyset, \quad \delta(q, b) = \{r\}, \quad \delta(r, a) = \delta(r, b) = \{r\}.$$

List the reachable DFA states, their transitions, and the final DFA states.

[12 marks]

- (b) Give regular expressions over $\{a, b\}$ for:

[10 marks]

- (i) words with an odd number of bs ;
- (ii) words containing substring aba ;
- (iii) words not ending in a ;
- (iv) words containing exactly two as ;
- (v) words where every b is immediately followed by an a .

- (c) Explain why the concatenation of two regular languages is regular.

[3 marks]**Question 3 - LL(1) parsing****[25 marks]**

Consider the grammar:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow a \mid (E) \end{aligned}$$

- (a) Explain why the left-recursive grammar $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$ is unsuitable for naive recursive descent, and why the grammar above avoids that problem. **[4 marks]**
- (b) Give the lookahead sets for the alternatives of E' , T' , and F . Use $\$$ for end of input. **[8 marks]**
- (c) Trace the recursive-descent parse of $a + a * a$, showing the main calls and where $+$ and $*$ are consumed. **[8 marks]**
- (d) Write pseudocode for either $parseT'$ or $parseF$. **[5 marks]**

Question 4 - PDAs and Turing machines**[25 marks]**

- (a) Describe a PDA recognising $\{a^n b^n \mid n \geq 0\}$. Your answer should include stack behaviour and accepting condition. **[7 marks]**
- (b) Trace your PDA on $aaabbb$. **[3 marks]**
- (c) Explain why nondeterminism is useful for recognising palindromes with a PDA. **[5 marks]**
- (d) Give a high-level description of a Turing machine deciding $\{a^n b^n c^n \mid n \geq 0\}$. **[6 marks]**
- (e) Explain the difference between accepting a language and deciding a language for Turing machines. **[4 marks]**

Practice Paper C - Pumping and Halting Mock

Time allowed: 2 hours. Total: 100 marks.

Answer all questions.

Question 1 - Multiple choice / True-False [25 marks]

1. Languages and Kleene algebra [5 marks]

- (a) A language over Σ is a subset of $Word(\Sigma)$.
- (b) $\square$ denotes the empty language.
- (c) $\{\varepsilon\}$ is the neutral element for language concatenation.
- (d) $\emptyset$ is the neutral element for union.
- (e) L^* contains ε for every language L .

2. Pumping lemma [5 marks]

- (a) The pumping lemma is normally used to prove that languages are regular.
- (b) If L is regular, then every sufficiently long word in L has a pumpable split.
- (c) If a language satisfies the pumping conclusion, it must be regular.
- (d) A standard non-regularity proof assumes regularity and derives a contradiction.
- (e) No finite language can satisfy the pumping lemma.

3. CFG facts [5 marks]

- (a) In a CFG, the left-hand side of a production is a single non-terminal.
- (b) Every regular language is context-free.
- (c) $\{a^n b^n \mid n \geq 0\}$ is context-free.
- (d) $\{a^n b^n c^n \mid n \geq 0\}$ is context-free.
- (e) Ambiguity means some word has two distinct parse trees.

4. Turing-machine facts [5 marks]

- (a) A Turing machine may loop forever on some inputs.
- (b) A decider halts on every input.
- (c) A recursively enumerable language is accepted by some TM, but no-instances may loop.
- (d) The complement of every recursively enumerable language is recursively enumerable.
- (e) If both L and $\bar{L}$ are recursively enumerable, then L is decidable.

5. Hierarchy facts [5 marks]

- (a) Regular languages are strictly contained in context-free languages.
- (b) Every context-free language is decidable.
- (c) Context-sensitive languages correspond to linear-bounded automata.
- (d) Type 0 languages are exactly the decidable languages.
- (e) Deterministic and nondeterministic Turing machines recognise the same languages.

Question 2 - Pumping lemma and regularity [25 marks]

- (a) Prove using the pumping lemma that $L = \{a^n b^n \mid n \geq 0\}$ is not regular. [10 marks]

- (b) Prove using the pumping lemma that $K = \{a^n b^{2n} \mid n \geq 0\}$ is not regular. [7 marks]
- (c) Let $M = \{w \in \{a, b\}^* \mid \#a(w) \equiv \#b(w) \pmod{3}\}$. Give finite-state evidence that M is regular. [5 marks]
- (d) Explain why the pumping lemma cannot by itself prove that a language is regular. [3 marks]

Question 3 - Palindrome CFGs and PDAs**[25 marks]**

Let P be the grammar over $\{a, b\}$:

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid \varepsilon.$$

- (a) Give derivations for $abba$ and aba . [6 marks]
- (b) Explain why every word generated by P is a palindrome. [5 marks]
- (c) Explain the difference between two derivation orders and true ambiguity. [5 marks]
- (d) Describe an NPDA for palindromes over $\{a, b\}$. [5 marks]
- (e) Why is the midpoint issue a problem for deterministic PDAs? [4 marks]

Question 4 - Halting problem and hierarchy**[25 marks]**

- (a) Explain how the Halting Problem can be treated as a language over a binary alphabet. [5 marks]
- (b) Explain why the Halting Problem is recursively enumerable. [5 marks]
- (c) Give the diagonalisation idea showing that the Halting Problem is not decidable. [8 marks]
- (d) Explain why $\{a^n b^n c^n \mid n \geq 0\}$ is a good example of a context-sensitive but not context-free language. [4 marks]
- (e) State the relationship between deterministic and nondeterministic Turing machines in terms of language recognition. [3 marks]

Practice Paper D - Synthesis Mock

Time allowed: 2 hours. Total: 100 marks.

Answer all questions.

Question 1 - Multiple choice / True-False [25 marks]

1. DFA E over $\{0, 1\}$ [5 marks]

DFA E has states q_0, q_1 , start/final state q_0 , and transitions that flip state on input 1 and stay in the same state on input 0.

- (a) E is a DFA.
- (b) $101 \in L(E)$.
- (c) $111 \in L(E)$.
- (d) E accepts exactly the words of even length.
- (e) $000 \in L(E)$.

2. NFA F over $\{0, 1\}$ [5 marks]

NFA F has states s, t, u , start s , final u , transitions $s \xrightarrow{0,1} s$, $s \xrightarrow{0} t$, $t \xrightarrow{1} u$, and $u \xrightarrow{0,1} u$.

- (a) F is nondeterministic.
- (b) F accepts every word containing substring 01.
- (c) $0010 \in L(F)$.
- (d) $111 \in L(F)$.
- (e) F's language is regular.

3. Regularity and closure [5 marks]

- (a) $\{a^n b^m \mid n, m \geq 0\}$ is regular.
- (b) $\{a^n b^n c^m \mid n, m \geq 0\}$ is regular.
- (c) The union of two regular languages is regular.
- (d) The intersection of a regular language and a non-regular language is always non-regular.
- (e) The complement of a regular language is regular.

4. CFG membership [5 marks]

Let H have productions

$$S \rightarrow AB, \quad A \rightarrow aA \mid \varepsilon, \quad B \rightarrow bBc \mid \varepsilon.$$

Which words are in $L(H)$?

- (a) aa
- (b) $aabbcc$
- (c) $abbcc$
- (d) $abc bc$
- (e) $bbcc$

5. Mixed theory [5 marks]

- (a) Every LL(1) grammar is unambiguous.
- (b) Every unambiguous CFG is LL(1).
- (c) Turing machines can use tape-only symbols to mark progress.
- (d) $\{a^n b^n c^n \mid n \geq 0\}$ is decidable.
- (e) Decidable languages form one of the four proper Chomsky hierarchy levels.

Question 2 - Regular expressions and closure reasoning**[25 marks]**Let $\Sigma = \{a, b, c\}$. Give regular expressions for:**[15 marks]**

- (a) words with at most one b ;
- (b) words where the number of a s is divisible by 3;
- (c) words ending in ca ;
- (d) words consisting of some c s followed by any mixture of a s and b s;
- (e) words not containing substring ac .

Then answer the following.

[10 marks]

- (i) If L is regular, explain why $\Sigma^* \setminus L$ is regular. **[3 marks]**
- (ii) If L is regular and K is not regular, must $L \cup K$ be non-regular? Justify. **[3 marks]**
- (iii) Give a closure-based proof that if $\bar{L}$ is regular then L is regular. **[2 marks]**
- (iv) Explain the difference between $\emptyset$ and $\{\varepsilon\}$. **[2 marks]**

Question 3 - Regular-expression grammar ambiguity**[25 marks]**Consider the grammar R for regular-expression syntax over terminals $a, b, c, +, \cdot, *, (,)$:

$$R \rightarrow R + R \mid R \cdot R \mid R^* \mid (R) \mid a \mid b \mid c \mid \varepsilon \mid \emptyset.$$

- (a) Give a derivation of the token string $(a + b) * \cdot c$. **[10 marks]**
- (b) Show that the grammar is ambiguous using the word $a + b \cdot c$. **[6 marks]**
- (c) Give an unambiguous grammar that enforces $*$ before $\cdot$ before $+$. **[6 marks]**
- (d) Explain why precedence and associativity are related but not identical issues. **[3 marks]**

Question 4 - Big-picture theory synthesis**[25 marks]**

- (a) Compare finite automata, pushdown automata, and Turing machines in terms of memory and language power. **[7 marks]**
- (b) Match each language class with a recogniser and a generator: regular, context-free, context-sensitive, recursively enumerable. **[6 marks]**
- (c) Explain why decidable languages are closed under complement, but recursively enumerable languages are not necessarily closed under complement. **[6 marks]**
- (d) Give one example language for each hierarchy level and one sentence explaining why it belongs there. **[6 marks]**

Detailed Mark Schemes

For regex, automata, grammar, parser, and proof questions, the answers below are representative. Award full credit for any equivalent correct construction or argument.

Mark Scheme: Practice Paper A

Question 1

- 1.1 A1: (a) T, (b) F, (c) T, (d) F, (e) T.
 1.2 A2: (a) T, (b) F, (c) T, (d) F, (e) T.
 1.3 Theory: (a) T, (b) F, (c) T, (d) F, (e) T.
 1.4 CFG G_1 generates $a^n c^m b^n$. Answers: (a) T, (b) T, (c) T, (d) F, (e) T.
 1.5 Halting: (a) T, (b) F, (c) T, (d) F, (e) T.

Question 2

Regex examples:

- (a) c^* .
 (b) $(a + b + c)^* bc(a + b + c)^*$.
 (c) $(b + c)^*(a(b + c)^* a(b + c)^*)^*$.
 (d) $a(a + b + c)^* b$.
 (e) $(a + b)^*$.
 (f) $(a + c)^* b(a + c)^*$.
 (g) Let $D = (a + c)$. One answer is $D^*(bDD^*)^*(\varepsilon + b)$.
 (h) $(a + b + c)(a + b + c)(a + b + c)^*$.
 (i) $a^* b^* c^*$.
 (j) $(a + b + c)^*(abc + ac)$.

Regularity: (i) regular, track parities of a and b with four states. (ii) not regular, classic pumping lemma example. (iii) regular, finite language. (iv) regular, complement of finite/regular language. (v) not regular over an alphabet with at least two symbols; repeated-word language requires unbounded comparison.

Question 3

- (a) One derivation:

$$E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E \Rightarrow a + a * E \Rightarrow a + a * a.$$

Award credit for any valid derivation.

- (b) Ambiguity: the word $a + a * a$ has two structures: $a + (a * a)$ and $(a + a) * a$. These correspond to different root productions, so different parse trees.
 (c) Standard unambiguous grammar:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow a \mid (E).$$

An LL(1)-style right-recursive variant also earns full credit.

- (d) F handles atoms/brackets, T builds products from factors, and E builds sums from terms. Since $*$ occurs inside T and $+$ only at the E level, multiplication binds tighter.

Question 4

- (a) Type 3: regular, DFA/NFA, regular grammar. Type 2: context-free, PDA, CFG. Type 1: context-sensitive, LBA, context-sensitive grammar. Type 0: recursively enumerable, TM, unrestricted grammar.
 (b) Examples: $(a + b)^*$ regular; $\{a^n b^n \mid n \geq 0\}$ CFL not regular; $\{a^n b^n c^n \mid n \geq 0\}$ context-sensitive not CFL; Halting Problem r.e. but undecidable.
 (c) Decidable: some TM halts on every input and accepts exactly the language. Recursively enumerable: some TM accepts yes-instances but may loop on no-instances.

- (d) Chomsky levels classify grammar families with clean generative restrictions. Decidable languages sit between context-sensitive and recursively enumerable, but are not one of the four standard grammar-defined levels.
- (e) TMs have an unbounded rewritable tape and can move both left and right, while PDAs only have stack access to the top symbol.

Mark Scheme: Practice Paper B

Question 1

- 1.1 D: (a) T, (b) T, (c) F, (d) T, (e) F.
 1.2 C: (a) T, (b) T, (c) T, (d) F, (e) F.
 1.3 Operations: (a) T, (b) T, (c) T, (d) F, (e) F.
 1.4 PDA: (a) T, (b) T, (c) F, (d) T, (e) F.
 1.5 Parser: (a) T, (b) T, (c) T, (d) F, (e) T.

Question 2

- (a) Reachable subset states: $\{p\}$, $\{p, q\}$, $\{p, r\}$, $\{p, q, r\}$. Transitions:

	a	b
$\{p\}$	$\{p, q\}$	$\{p\}$
$\{p, q\}$	$\{p, q\}$	$\{p, r\}$
$\{p, r\}$	$\{p, q, r\}$	$\{p, r\}$
$\{p, q, r\}$	$\{p, q, r\}$	$\{p, r\}$

Final DFA states are all subsets containing r : $\{p, r\}$ and $\{p, q, r\}$ among the reachable states.

- (b) Regex examples:
 (i) $a^*b(a^*ba^*b)^*a^*$.
 (ii) $(a + b)^*aba(a + b)^*$.
 (iii) $(a + b)^*b + \varepsilon$.
 (iv) $b^*ab^*ab^*$.
 (v) $(a + ba)^*$.
 (c) If L_1, L_2 are regular, choose NFAs for them and connect final states of the first to initial states of the second in the usual concatenation construction; equivalently, if L_1, L_2 have regexes, concatenate the regexes.

Question 3

- (a) Left recursion like $E \rightarrow E + T$ makes $\text{parse}E$ call $\text{parse}E$ before consuming input. The transformed grammar uses right-recursive tails E' and T' , so parsing consumes a T or F before recurring.
 (b) Lookahead sets: $E' \rightarrow +TE'$ has $\{+\}$; $E' \rightarrow \varepsilon$ has $\{), \$\}$. $T' \rightarrow *FT'$ has $\{*\}$; $T' \rightarrow \varepsilon$ has $\{+,), \$\}$. $F \rightarrow a$ has $\{a\}$; $F \rightarrow (E)$ has $\{(\}$.
 (c) Trace outline: $\text{parse}E \rightarrow \text{parse}T \rightarrow \text{parse}F$ consumes first a ; $\text{parse}T'$ sees $+$ and returns; $\text{parse}E'$ consumes $+$, calls $\text{parse}T$; inside that $\text{parse}F$ consumes second a , $\text{parse}T'$ consumes $*$ and then $\text{parse}F$ consumes final a ; remaining T' and E' take ε at end marker.
 (d) Example pseudocode for $\text{parse}T'$:

```

parseTprime():
  if curr == "*":
    next(); parseF(); parseTprime()
  else if curr in {"+", ")", "$"}:
    return
  else:
    error()
  
```

Equivalent $\text{parse}F$ pseudocode also earns credit.

Question 4

- (a) Push one stack marker for each a . When the first b is read, switch to popping one marker per b . Accept when input is exhausted and the stack has returned to its bottom marker. Include a special transition for $n = 0$ if accepting ε .
- (b) Trace: start $[\#]$; after aaa stack has $XXX\#$; read three b s and pop X, X, X ; input empty and stack $\#$, then accept.
- (c) For palindromes, the machine must choose the midpoint. An NPDA can guess the midpoint nondeterministically and then compare the second half with the stack.
- (d) TM for $a^n b^n c^n$: repeatedly mark the leftmost unmarked a , then the next unmarked b , then the next unmarked c ; return left and repeat. When no unmarked a remains, verify no unmarked b or c remains and accept; otherwise get stuck/reject.
- (e) Accepting allows looping on no-instances. Deciding requires halting on every input, accepting yes-instances and rejecting/stopping on no-instances.

Mark Scheme: Practice Paper C

Question 1

- 1.1 Languages: (a) T, (b) F, (c) T, (d) T, (e) T.
- 1.2 Pumping: (a) F, (b) T, (c) F, (d) T, (e) F.
- 1.3 CFG: (a) T, (b) T, (c) T, (d) F, (e) T.
- 1.4 TM: (a) T, (b) T, (c) T, (d) F, (e) T.
- 1.5 Hierarchy: (a) T, (b) T, (c) T, (d) F, (e) T.

Question 2

- (a) Assume L regular with pumping length p . Choose $w = a^p b^p$. For any split $w = xyz$ with $|xy| \leq p$ and $|y| \geq 1$, y consists only of as . Pump down: xz has fewer as than bs , so $xz \notin L$, contradiction.
- (b) Assume K regular with pumping length p . Choose $w = a^p b^{2p}$. Again y consists only of as . Pump down gives fewer than p as but still $2p$ bs , so the number of bs is not twice the number of as . Contradiction.
- (c) Use a DFA with states $(i, j) \in \{0, 1, 2\}^2$ recording $\#a \bmod 3$ and $\#b \bmod 3$. Start $(0, 0)$; on a increment first coordinate mod 3; on b increment second. Accept states $(0, 0), (1, 1), (2, 2)$.
- (d) The pumping lemma gives a necessary condition for regularity, not a sufficient one. Some non-regular languages may still satisfy the pumping-style conclusion, so satisfying it does not prove regularity.

Question 3

- (a) $abba$: $S \Rightarrow aSa \Rightarrow abSba \Rightarrow abba$ using $S \rightarrow \varepsilon$ in the middle. aba : $S \Rightarrow aSa \Rightarrow aba$ using $S \rightarrow b$.
- (b) Each recursive rule adds matching symbols to both ends. The base cases ε, a, b are palindromes. By induction, all generated words are palindromes.
- (c) Two derivation orders can apply productions in different sequences while producing the same parse tree. True ambiguity requires two distinct parse trees/structures for the same terminal word.
- (d) NPDA idea: push input symbols while in the first phase; nondeterministically guess the midpoint, either by an ε -move for even length or by consuming one middle symbol for odd length; then pop and match the rest of the input against the stack; accept at bottom marker with input exhausted.
- (e) A DPDA has no nondeterministic midpoint guess. Before seeing the rest of the input, it cannot know where to switch from pushing to popping for arbitrary palindromes.

Question 4

- (a) Encode each TM and input word as binary strings, pair them with a delimiter/encoding scheme, and define $HALT = \{\langle M, w \rangle \mid M \text{ halts on } w\}$.
- (b) A universal TM can simulate M on w . If the simulation halts, accept. If M never halts, the simulation runs forever. Therefore $HALT$ is r.e.
- (c) Suppose a decider H for $HALT$ exists. Build a diagonal machine D that on input x asks whether machine x halts on input x ; if yes, D loops, and if no, D halts. Running D on its own encoding creates the contradiction: D halts iff it does not halt.
- (d) $a^n b^n c^n$ requires comparing three counts, beyond a single-stack PDA, but can be handled with bounded tape by marking matching triples; hence it is context-sensitive but not context-free.
- (e) NDTMs and DTMs recognise the same class of languages; nondeterminism may affect complexity, not recognisability.

Mark Scheme: Practice Paper D

Question 1

- 1.1 E: (a) T, (b) T, (c) F, (d) F, (e) T.
 1.2 F: (a) T, (b) T, (c) T, (d) F, (e) T.
 1.3 Closure: (a) T, (b) F, (c) T, (d) F, (e) T.
 1.4 CFG H generates $a^i b^j c^j$. Answers: (a) T, (b) T, (c) T, (d) F, (e) T.
 1.5 Mixed: (a) T, (b) F, (c) T, (d) T, (e) F.

Question 2

Regex examples:

- (a) $(a + c)^* + (a + c)^* b (a + c)^*$.
 (b) $(b + c)^* (a(b + c)^* a(b + c)^* a(b + c)^*)^*$.
 (c) $(a + b + c)^* ca$.
 (d) $c^* (a + b)^*$.
 (e) $(b + c)^* (aa^* b (b + c)^*)^* a^*$.

Closure reasoning:

- (i) If L is recognised by a complete DFA, swap final and non-final states to recognise the complement.
 (ii) No. For example, if $L = \Sigma^*$ and K is any non-regular language over Σ , then $L \cup K = \Sigma^*$, which is regular.
 (iii) If $\bar{L}$ is regular, then its complement $\Sigma^* \setminus \bar{L} = L$ is regular by closure under complement.
 (iv) $\emptyset$ contains no words. $\{\varepsilon\}$ contains exactly one word: the empty word.

Question 3

- (a) One derivation:

$$R \Rightarrow R \cdot R \Rightarrow R * \cdot R \Rightarrow (R) * \cdot R \Rightarrow (R + R) * \cdot R \Rightarrow (a + R) * \cdot R \Rightarrow (a + b) * \cdot R \Rightarrow (a + b) * \cdot c.$$

- (b) $a + b \cdot c$ can be parsed as $a + (b \cdot c)$ or $(a + b) \cdot c$. These are different parse trees with different root operators.
 (c) One precedence grammar:

$$\begin{aligned} R &\rightarrow PR' \\ R' &\rightarrow +PR' \mid \varepsilon \\ P &\rightarrow SP' \\ P' &\rightarrow \cdot SP' \mid \varepsilon \\ S &\rightarrow AS' \\ S' &\rightarrow *S' \mid \varepsilon \\ A &\rightarrow (R) \mid a \mid b \mid c \mid \varepsilon \mid \emptyset \end{aligned}$$

Equivalent left- or right-recursive precedence grammars also earn credit.

- (d) Precedence decides which operator binds more tightly between different operators. Associativity decides how repeated uses of the same operator group, e.g. $a + b + c$ as $(a + b) + c$ or $a + (b + c)$.

Question 4

- (a) Finite automata have only finite control and recognise regular languages. PDAs add a stack and recognise context-free languages. TMs add an unbounded rewritable tape and recognise recursively enumerable languages, with deciders recognising decidable languages.
 (b) Regular: DFA/NFA and regular grammar. Context-free: PDA and CFG. Context-sensitive: LBA and context-sensitive grammar. Recursively enumerable: TM and unrestricted grammar.

- (c) Decidable languages are closed under complement: run the decider and swap accept/reject. R.e. languages are not necessarily closed under complement because the recogniser may loop on no-instances, leaving no reliable way to accept exactly the complement.
- (d) Examples: a^* is regular; $\{a^n b^n\}$ is CFL not regular; $\{a^n b^n c^n\}$ is context-sensitive not CFL; HALT is r.e. and undecidable/type 0.